

[← Back to Home](#)

# Project 5: Fun With Diffusion Models!

## Part A: The Power of Diffusion Models!

---

### Part 0: Setup

Drawing inspiration from the prompts given in the Huggingface Embedding Generators, I decided to try generating the embeddings for 16 prompts (including 'a high quality photo' and the null prompt '') that span different areas like artwork, nature, food/drink, etc.

Below are 3 prompts that I chose to generate images for: 'a watercolor painting of a cityscape', 'a bowl of beef noodle soup', 'a night sky'. I also used a random seed value = 42 this part and the following parts.

Interestingly, I felt that not all the images from using a higher num\_inference\_steps value at 150 were better than those from using a lower value at 20. That being said, maybe it's difficult to generalize this opinion because some of my prompts (especially when it comes to generating art like watercolors or oil paintings) may be subjective when it comes to determining quality and what makes an image better than another. I think the bowl of beef noodle soup prompt is higher quality at a higher step value because there's more detail/sharpness to the toppings in the soup, as well as a background object (likely a pot containing the soup) on top of the soup itself as the main image. However, for the watercolor painting of a cityscape, I feel like there's a little too much happening in the image (e.g. more city skylines in the background?) compared to the simplicity and neatness of the first image. Finally, I don't think there's too much difference in the night sky images, but the second one does look more like a high-res capture using some type of

advanced telescope and with more postprocessing coloring than the night sky image at 20 steps, but the first image is by no means bad to me either, considering the faster runtime.



Using num\_inference\_steps = 20



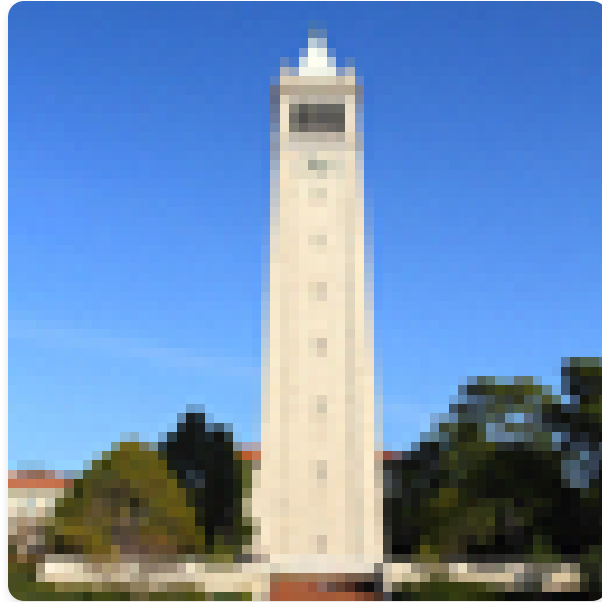
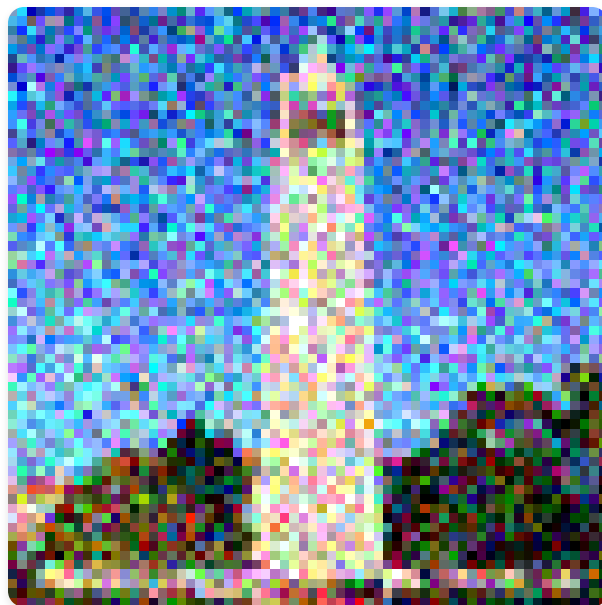
Using num\_inference\_steps = 150

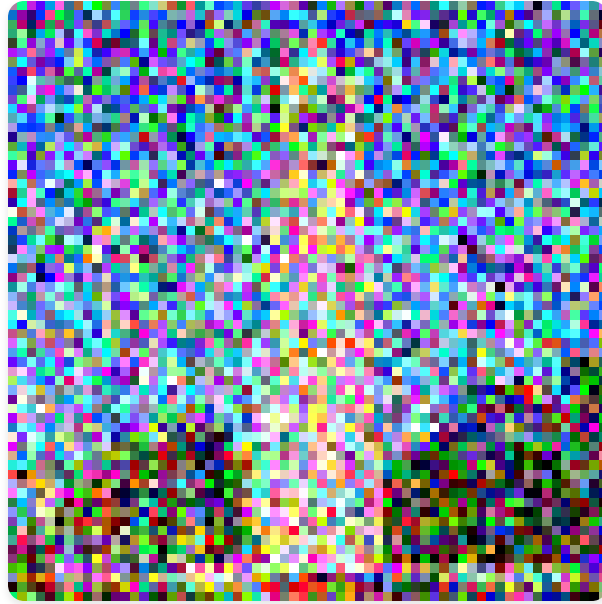
## Part 1: Sampling Loops

### 1.1 Implementing the Forward Process

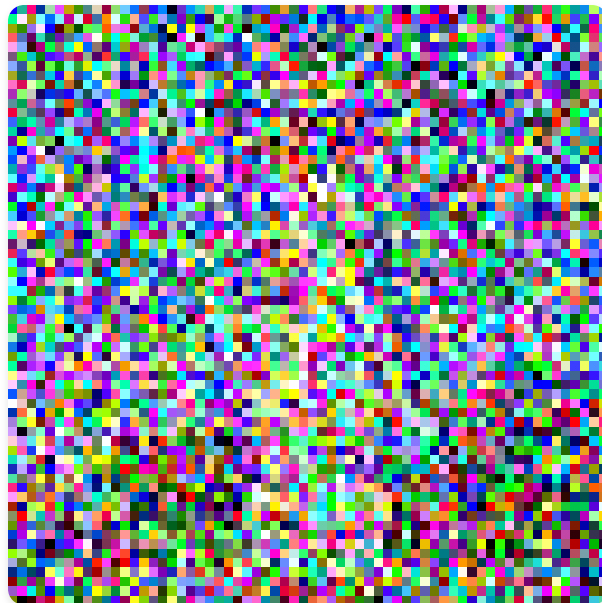
To implement the forward process for diffusion, I followed the spec's hints about indexing into the `alphas_cumprod` array at index `t` to retrieve the value of `alpha` at a specific timestep `t`. From there, we can also use the given

`torch.randn_like` function to sample noise from the image for the `epsilon` value. Below are images showing the forward diffusion process at the different time steps `t = 0, 250, 500, 750`. As expected, we can see that as `t` increases, the image becomes progressively noisier and less recognizable. For example, by `t = 750`, the original image is almost completely noise.

 $t = 0$  $t = 250$



$t = 500$



$t = 750$

## 1.2 Classical Denoising

To try denoising these images using classical methods, we may use Gaussian blur filtering as suggested by the spec. Here, we can leverage the

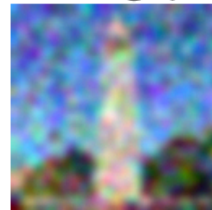
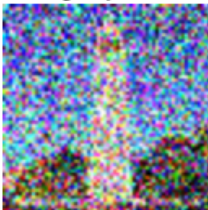


`torchvision.transforms.functional.gaussian_blur` function by calling it as `TF.gaussian_blur(noisy, 5, 1.5)` with the parameters for kernel size = 5 and sigma = 1.5. As predicted, the denoised image still appears rather noisy.

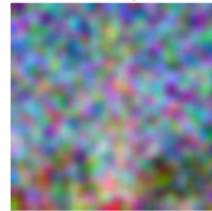
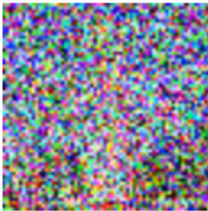
Noisy (t=250) Gaussian Blur Filtering (Denoised) (t=250)



Noisy (t=500) Gaussian Blur Filtering (Denoised) (t=500)



Noisy (t=750) Gaussian Blur Filtering (Denoised) (t=750)



Classical Denoising Result

### 1.3 One-Step Denoising

For one-step denoising, I first ran my `forward` process function to add noise to our image. Next, our pretrained denoiser `stage_1.unet` can estimate the noise in the new noisy image when we pass our noisy image through. Then, to remove the noise from the noisy image to obtain an estimate of the original image, we can follow formula A.2 but rewrite it to solve for our clean image  $x_0$ . Starting from formula A.2:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad \text{where } \epsilon \sim N(0, 1) \quad (\text{A.2})$$

We can rewrite this as:

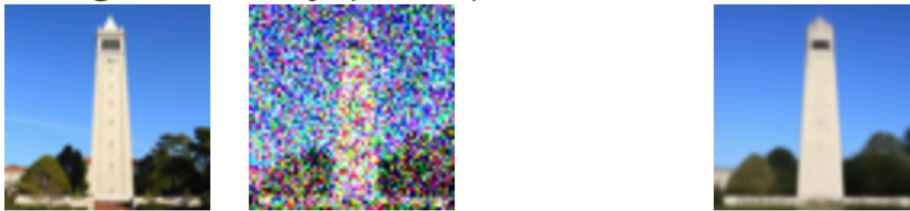
$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}$$

As the spec notes, I also did use the `.to(device)` and `.half()` functions to ensure my denoiser inputs matched the correct device and data types as the denoiser.

Original Noisy (t=250) UNet Denoised Estimate (t=250)



Original Noisy (t=500) UNet Denoised Estimate (t=500)



Original Noisy (t=750) UNet Denoised Estimate (t=750)



One-Step Denoising with stage\_1.unet

## 1.4 Iterative Denoising

Moving on from one-step denoising to iterative denoising, we can now repeatedly apply the denoiser to progressively denoise the image over multiple timesteps. First, I initialized a list of timesteps for the `strided_timesteps` variable starting from 990 with a stride of 30 until we reach 0. Next, I indexed into the `alphas_cumprod` array at each timestep to retrieve the corresponding `alpha_cumprod` and `alpha_cumprod_prev` values. Then, we can compute `alpha` by dividing `alpha_cumprod` by `alpha_cumprod_prev`. From there, we can find `beta` by calculating:  $1 - \alpha$ . Next, we can run our `stage_1.unet` denoiser to find the noise estimate that we use to calculate our clean image `x_0` like in our rewritten A.2 equation. With this, we can find our noisy image at the previous timestep `x_(t-1)` using formula A.3 from the spec:

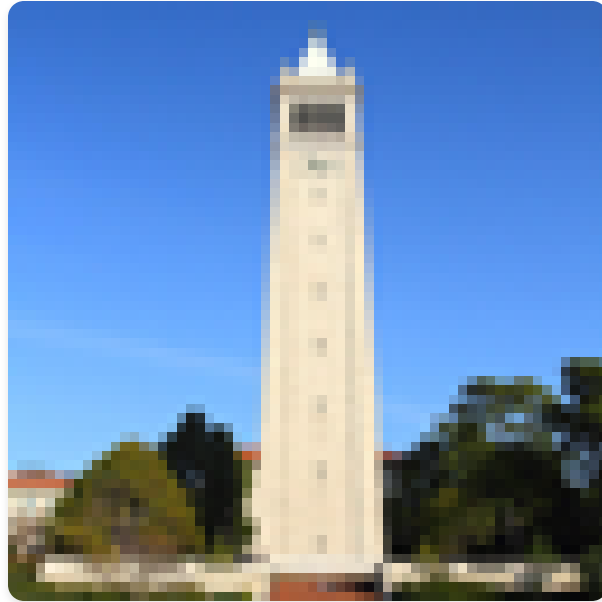
$$x_{t'} = \frac{\sqrt{\bar{\alpha}_{t'}}\beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t'})}{1 - \bar{\alpha}_t} x_t + v_\sigma \quad (\text{A.3})$$

Finally, we add variance to this image using our `add_variance` helper function at every timestep except for the last one.

Noisy Campanile at t = 90 Noisy Campanile at t = 240 Noisy Campanile at t = 390 Noisy Campanile at t = 540 Noisy Campanile at t = 690



Noisy Campanile every 5th loop of denoising



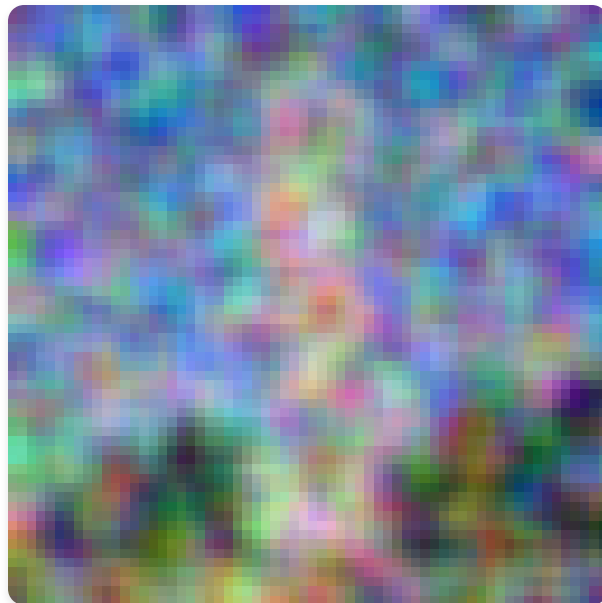
Original



Iteratively Denoised Campanile



One-Step Denoised Campanile



Gaussian Blurred Campanile

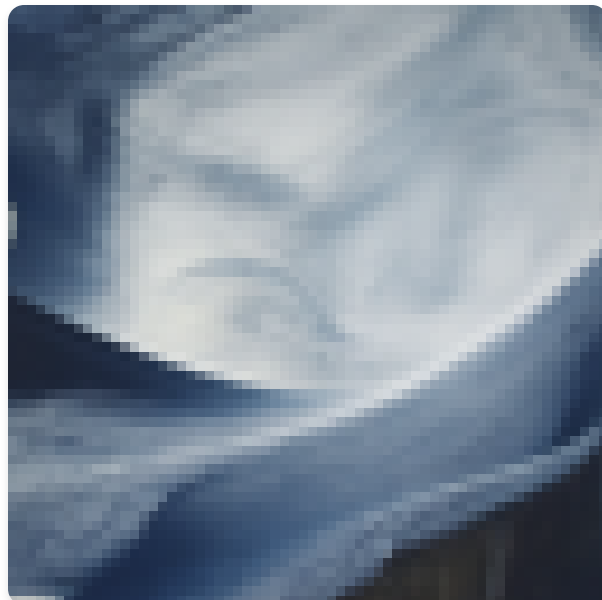
## 1.5 Diffusion Model Sampling

Beyond denoising an existing image, we can also generate new images from scratch by passing in a randomly sampled noise image into our

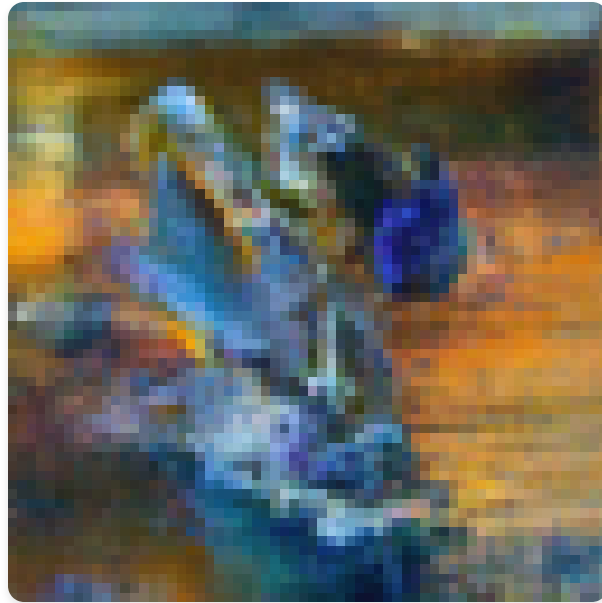
`iterative_denoise` function. This function will then iteratively denoise the random noise image over multiple timesteps to produce a final denoised image. Below are images showing the generated images from random noise after running the iterative denoising process for 5 samples using the prompt "a high quality photo".



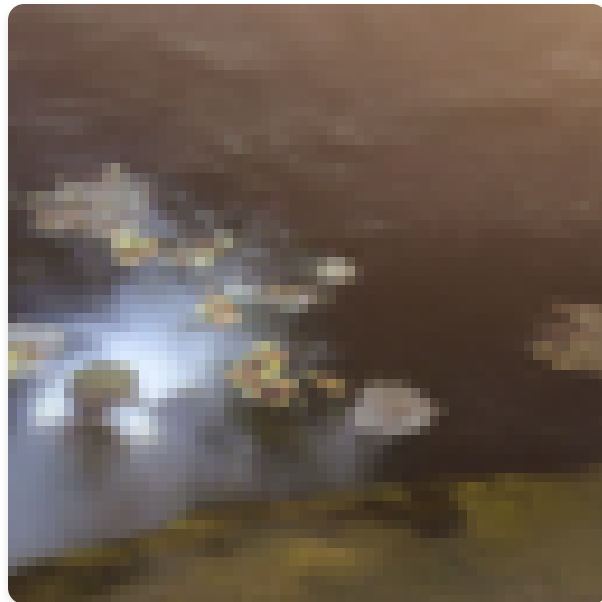
Sample 1



Sample 2



Sample 3



Sample 4



Sample 5

## 1.6 Classifier-Free Guidance (CFG)

In this next part, we improve our image quality (albeit at the expense of image diversity) by using CFG. Using "a high quality photo" as our conditional prompt embedding and a the null prompt "" as our unconditional prompt embedding, we run our UNet twice: once for both prompts. Then, using the noise estimates from our UNet, we can compute a new noise estimate by following the CFG formula:

$$\epsilon = \epsilon_u + \gamma(\epsilon_c - \epsilon_u) \quad (\text{A.4})$$

Then we use this new noise estimate when finding our clean image  $x_0$ . Below are 5 newly sampled images on the same prompt "a high quality photo" using our `iterative_denoise_cfg` function. I do notice an increase in image quality in terms of the detail as well as the subjects in the images being more well-defined than before. That being said, I can also understand what the spec meant by "at the expense" of image diversity, because now all 5 images seem to be photos of people for some reason.

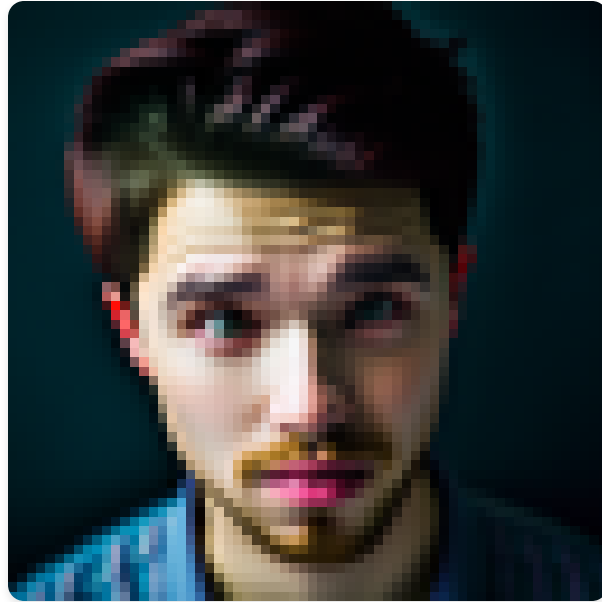




Sample 1



Sample 2



Sample 3



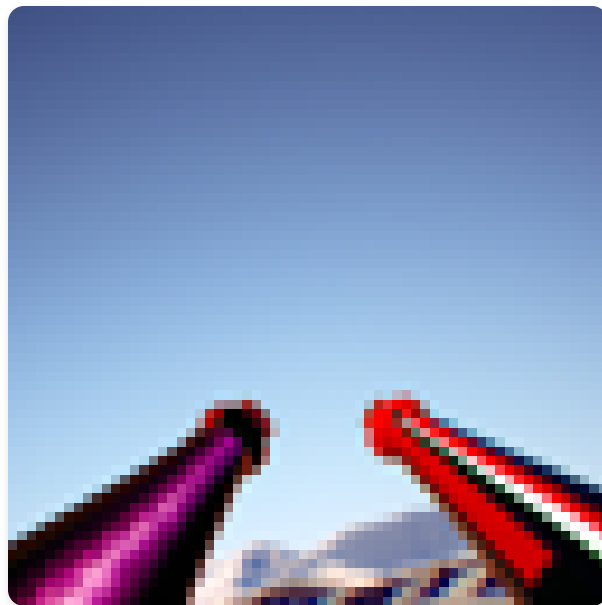
Sample 4

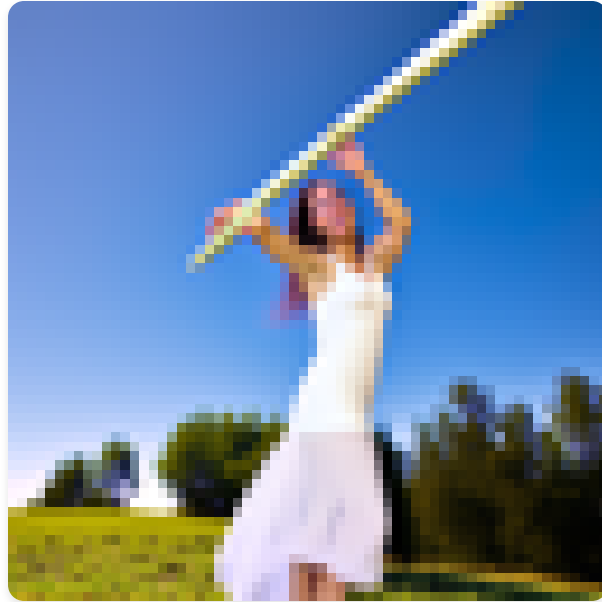
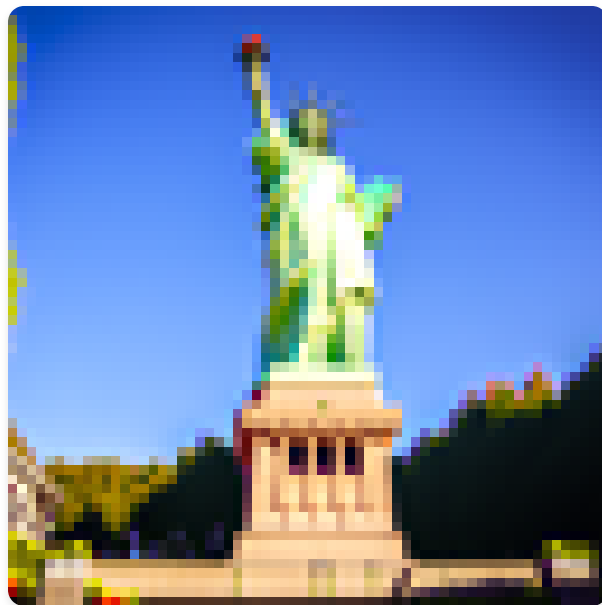


Sample 5

## 1.7 Image-to-image Translation

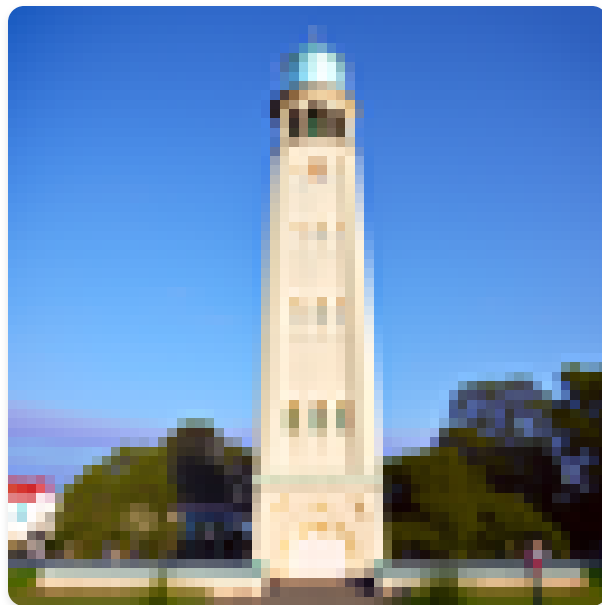
First, we run the forward diffusion process on our Campanile image, and then input this noisy image into our `iterative_denoise_cfg` function over iterations from [1, 3, 5, 7, 10, 20] steps. With each increasing step, we get closer and closer to the original Campanile image.

SDEdit with  $i_{\text{start}} = 1$ SDEdit with  $i_{\text{start}} = 3$

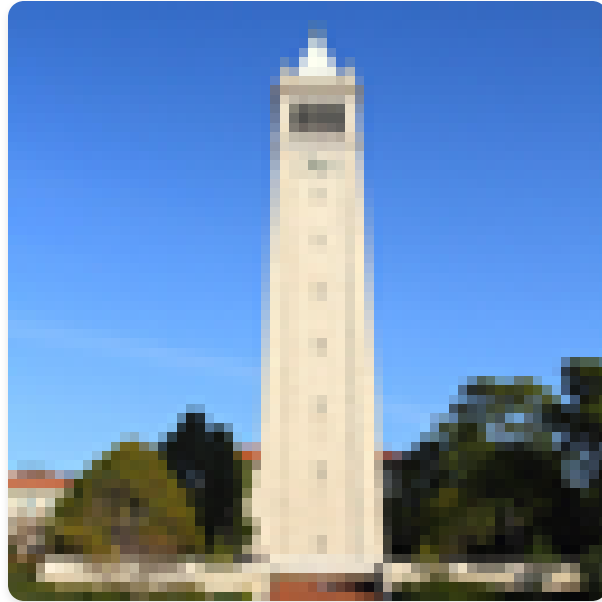
SDEdit with  $i\_start = 5$ SDEdit with  $i\_start = 7$



SDEdit with  $i_{\text{start}} = 10$

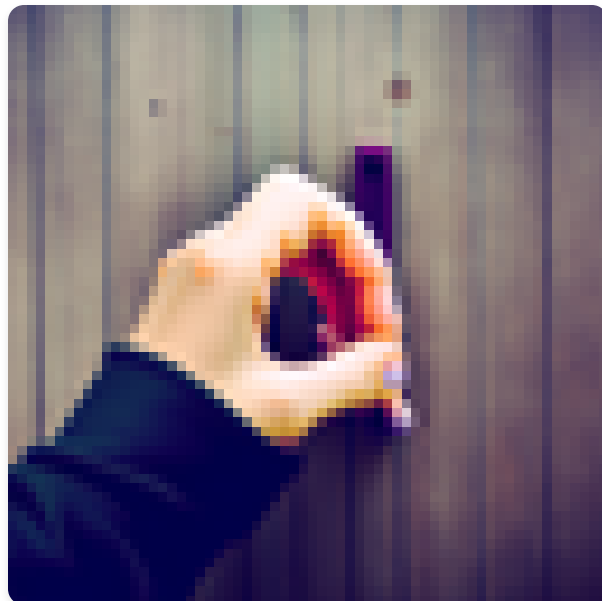


SDEdit with  $i_{\text{start}} = 20$



Campanile

I also tried this on 2 of my own images: a photo of Trouper the dog and one of my housemate's cat! Interestingly, for both images, I think the denoised outputs appeared to start to resemble the original image truly only around step 20, which is very cool to see such a big jump from all the possibilities on the natural image manifold reach the original clean image.



SDEdit with  $i\_start = 1$



SDEdit with  $i\_start = 3$

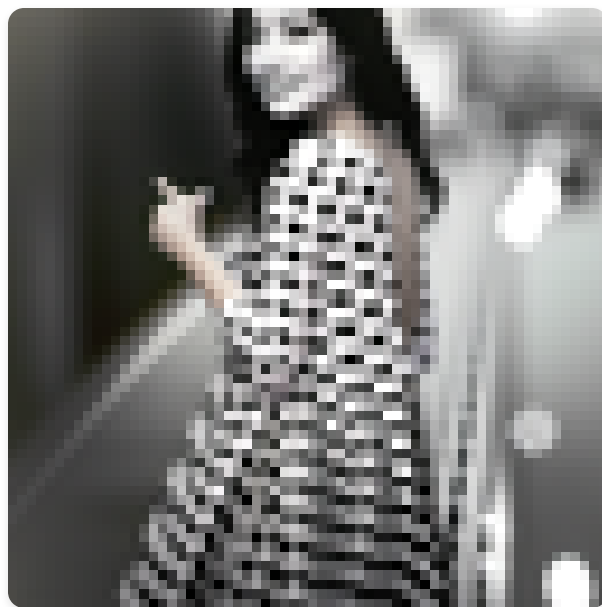


SDEdit with  $i\_start = 5$





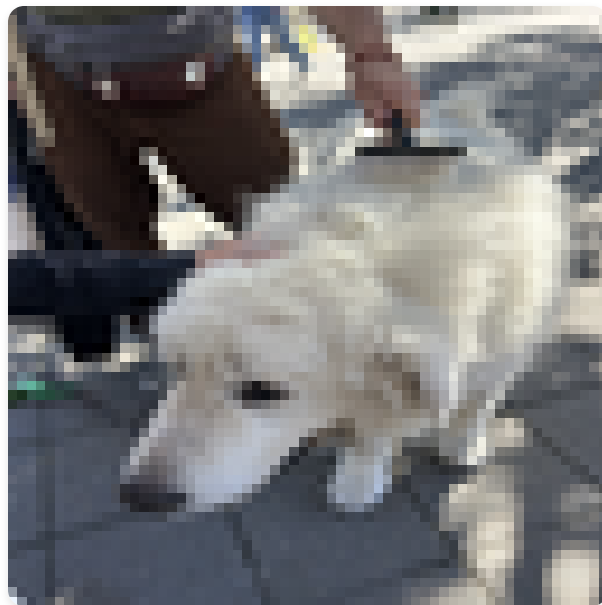
SDEdit with  $i_{\text{start}} = 7$



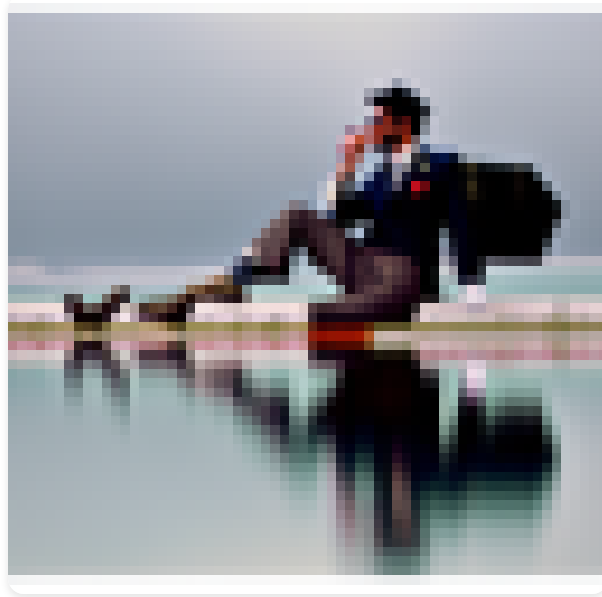
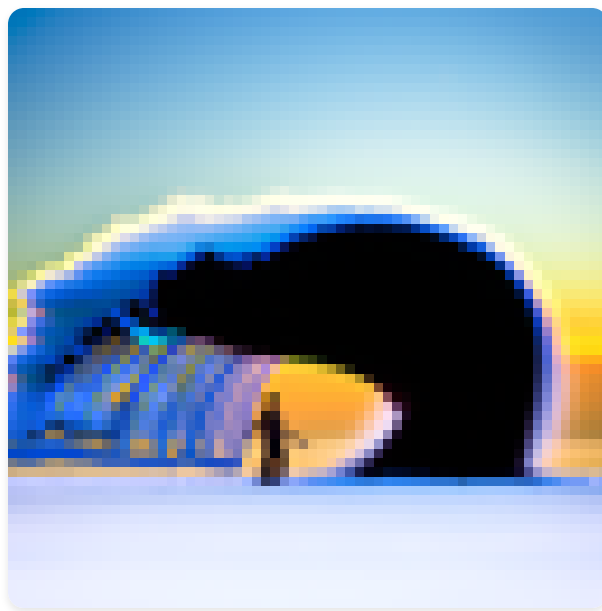
SDEdit with  $i_{\text{start}} = 10$



SDEdit with  $i_{\text{start}} = 20$

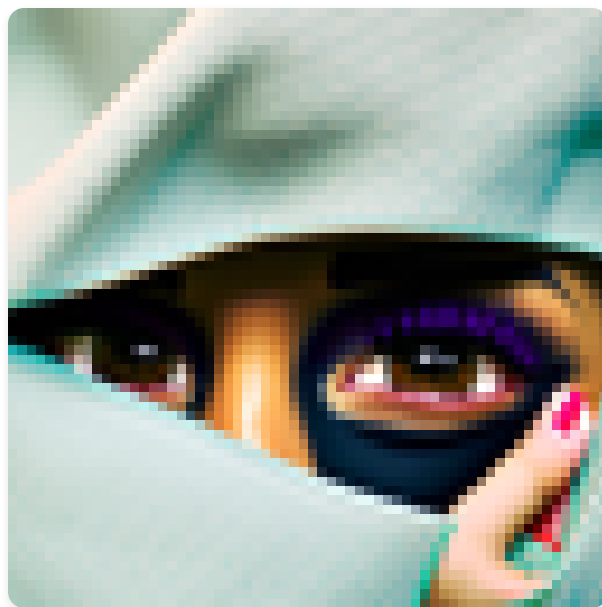


Trouper

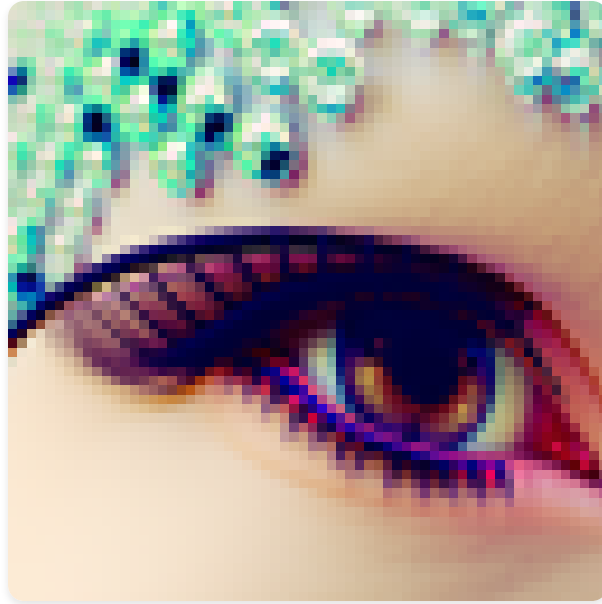
SDEdit with  $i_{\text{start}} = 1$ SDEdit with  $i_{\text{start}} = 3$



SDEdit with  $i_{\text{start}} = 5$



SDEdit with  $i_{\text{start}} = 7$



SDEdit with  $i\_start = 10$



SDEdit with  $i\_start = 20$



Happy (the cat!)

### 1.7.1 Editing Hand-Drawn and Web Images

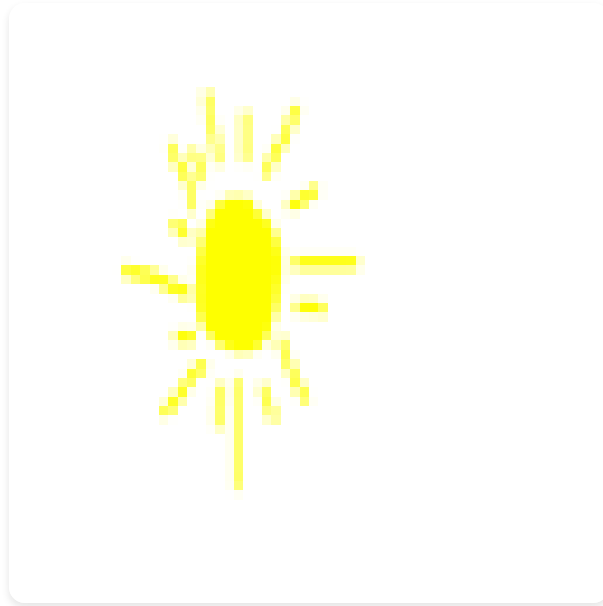
We can also apply this editing by leveraging diffusion noisiness and CFG denoising to nonrealistic images to see what "edits" our models may make to our images to determine the original clean image from a noisy input. Here, I try this on the a watercolor painting of an orchid, as well as a handrawn image of a heart and a sun.



Watercolor Orchid



Handrawn Heart



Handrawn Sun



Watercolor Orchid Edits



Handrawn Heart Edits

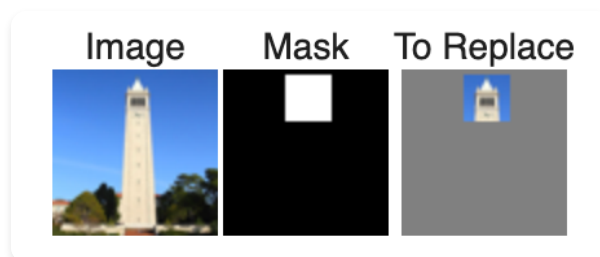




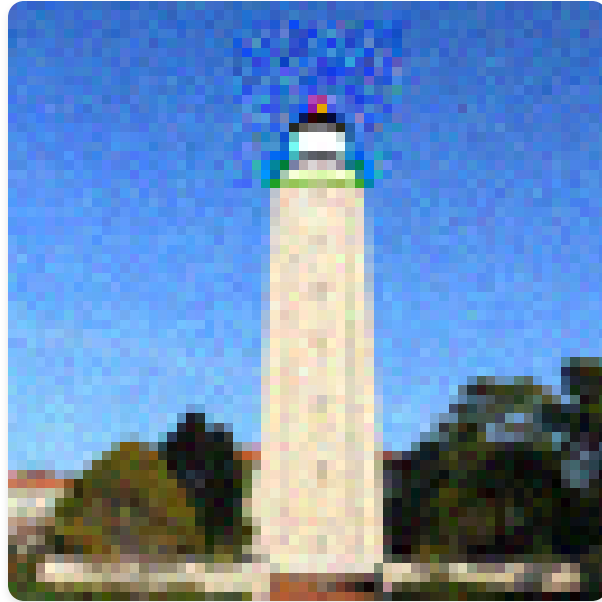
Handrawn Sun Edits

## 1.7.2 Inpainting

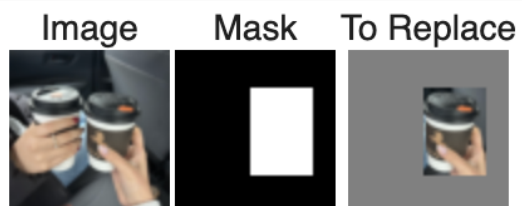
By including a binary mask, we can also selectively inpaint certain regions of an image while keeping other regions intact. To do this, I added the mask by following equation A.5 after running forward diffusion. Creating the mask itself took some trial and error to figure out the right indices for coffee and soup inpainted images, but the results were really interesting to see in terms of what the model could come up. I particularly liked the soup inpaint of having a photo on the table, which makes total sense in the context of the rest of the (unmasked) image. However, I was a bit confused by the output of the coffee inpaint because even after running many iterations, none of the outputs made that much sense. I wonder if it's because registering the hands holding the coffee cups in the original photo was too confusing or maybe not cropped at the best point given our mask is rectangular and didn't fully capture my hand as well as the entirety of my arm?



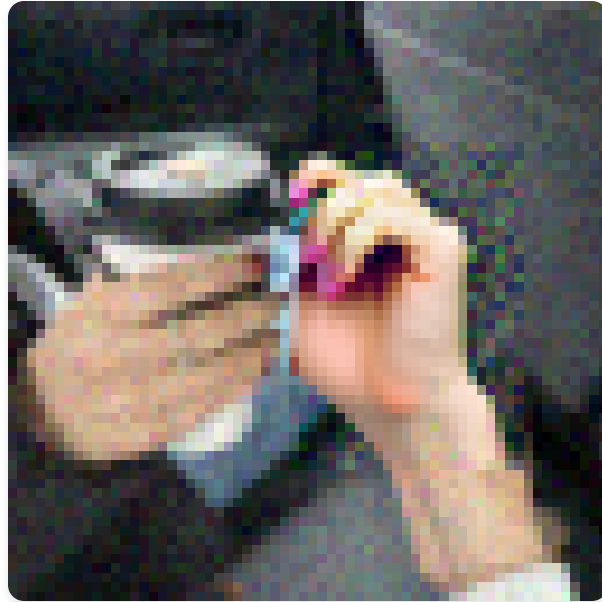
Campanile Masking



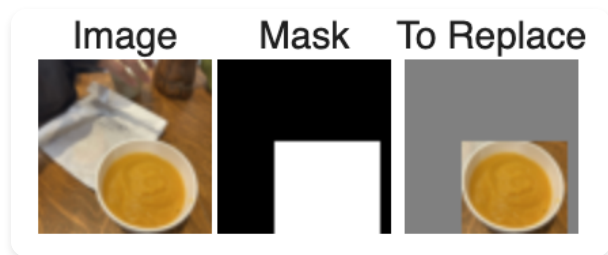
Inpainted Campanile



Coffee Masking



Inpainted Coffee



Soup Masking



Inpainted Soup

### 1.7.3 Text-Conditional Image-to-image Translation

Next, to guide our projection with a text prompt, I tried edits on the same set of 3 images (Campanile, coffees, and soup) but this time with the prompt of "a cup of hot chocolate". The results were pretty interesting! For the Campanile, I can see some more festive/decorative elements than there are in the original image. With the coffee image, there seems to be more intuition because the output image looks very natural! The disposable coffee containers now appear to be mugs with toppings like marshmallows. Finally, the bowl of soup also appears to have turned into a cup of coffee instead of the original image of butternut squash soup!



## Text-Conditioned Campanile



Text-Conditioned Coffee

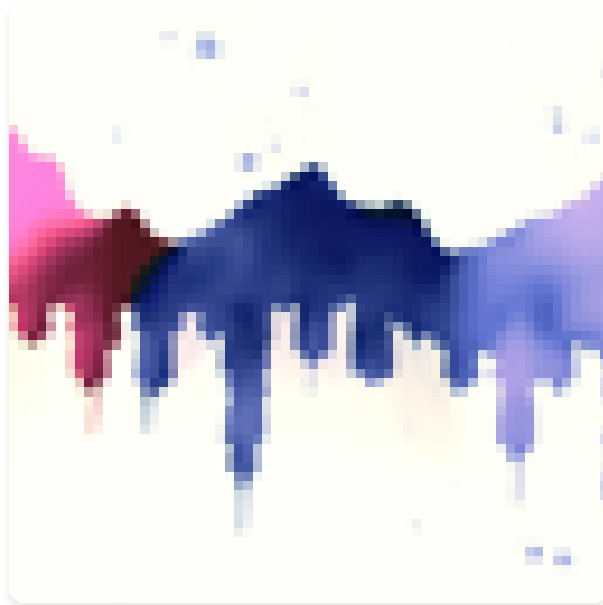


Text-Conditioned Soup

## 1.8 Visual Anagrams

To implement our `visual_anagrams` function, we can modify our code to be compatible with 2 prompt embeddings to determine the image when upright vs. when flipped. By calculating separate noise estimates for the upright image as well as the flipped image, we then flip the flipped noise estimate back to upright orientation, and then take the average of the two noise estimates to use for finding our clean image  $x_0$ . Below are 2 examples I tried: a watercolor painting of a cityscape and a mountain, as well as 'a photo of a golden retriever and a black cat' 'an oil painting of autumn on the east coast', both of which I was really

impressed by! The flipped oil painting is actually quite painterly in style in my opinion because it avoids harsh edges/strokes.



Watercolor Mountains



Watercolor Cityscape



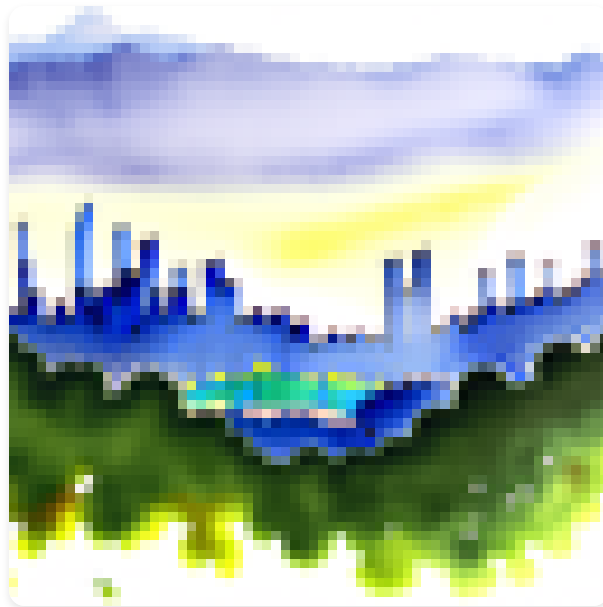
Golden Retriever and Black Cat



Autumn on East Coast

## 1.9 Hybrid Images

For another fun experiment combining images with diffusion models, we can use the `make_hybrids` function to essentially perform a high pass and low pass of 2 different image prompts (using a Gaussian blur) and then add the noise estimates from both images together to find our resulting image. Here, I tried this for a hybrid image of a watercolor painting of a mountain and a cityscape (again HAHA) where the mountain is the low pass image and the cityscape is the high pass image, and then a hybrid image of a baby grand piano (low pass) and a night sky (high pass).



Hybrid image of watercolor paintings of a mountain and a cityscape





Hybrid image of a baby grand piano and a night sky

## Part B: Flow Matching from Scratch!

### Part 1: Training a Single-Step Denoising UNet

#### 1.1 Implementing the UNet

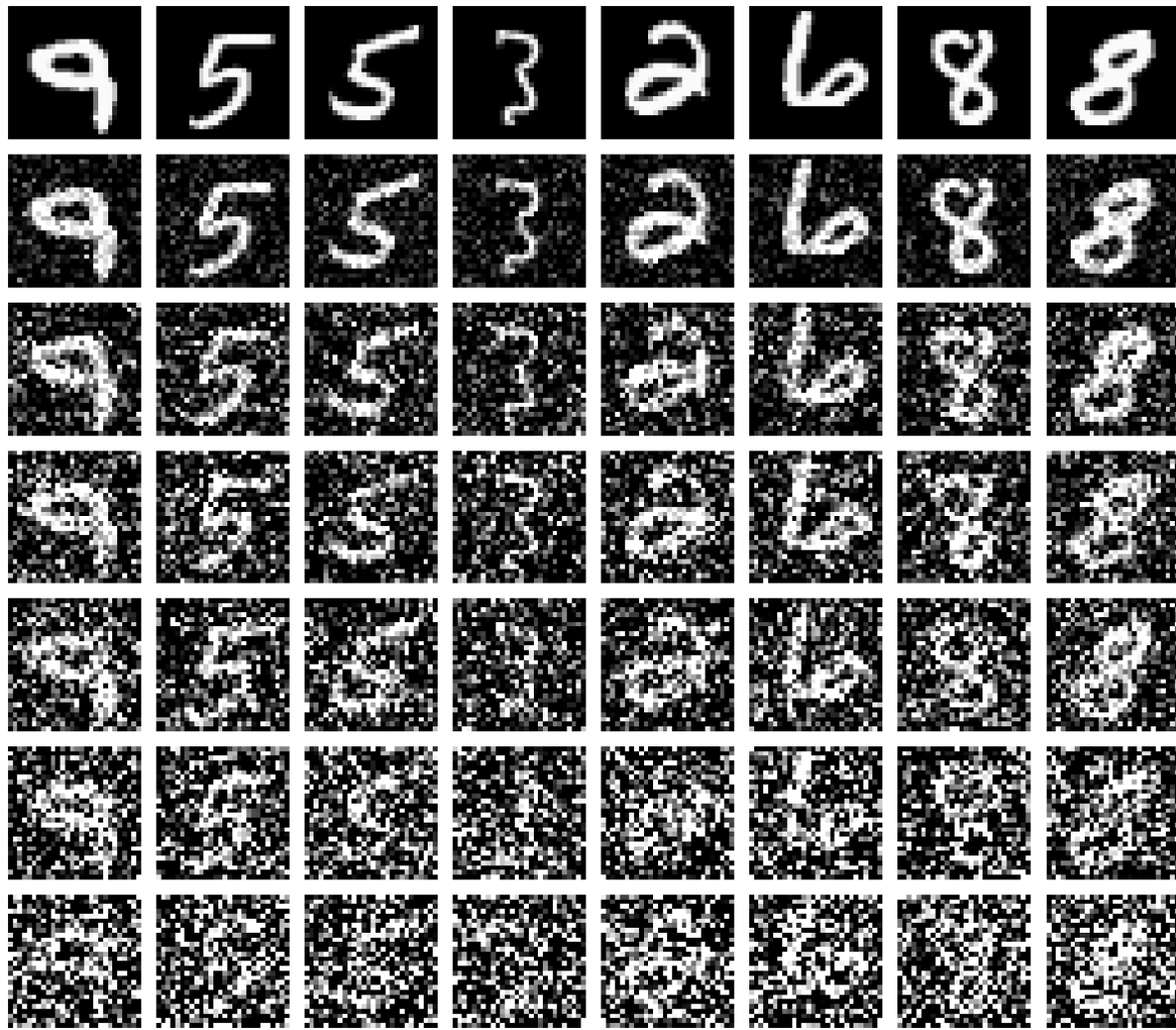
To implement our UNet, we first implement some individual operations like the Conv, DownConv, and UpConv operations to perform convolution, downsampling, and upsampling respectively. We also reshape the dimensions of our feature maps by using Flatten to convert 7x7 to 1x1, and vice versa Unflatten to convert 1x1 back up to 7x7. Then we combine these operations into blocks through ConvBlock, DownBlock, and UpBlock classes to create the building blocks for following our diagram of the UNet model.

Next, we implement our UnconditionalUNet by first passing our image through the convolutional layer, then continue downsampling the dimensions through 2 DownBlock calls. From here, we have dimensions at 7x7, so we use our flatten operation to convert to 1x1 dimensions. Then, we Unflatten this back to 7x7 and

follow the upsampling path by making 2 UpBlock calls. Finally, we pass this through a final convolutional layer to get our output image. In the forward method, we also concatenate corresponding encoder and decoder layers and then propagate upward following figure 1 from the spec.

## 1.2 Using the UNet to Train a Denoiser

Before training the UNet to remove noise, we first visualize the effect of adding varying levels of Gaussian noise to the original dataset images. We load our MNIST dataset and then define our list of noise levels to add to the images: [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]. As we can see in our grid visual, higher sigma values lead to more noise in the images, making them progressively harder to distinguish digits.



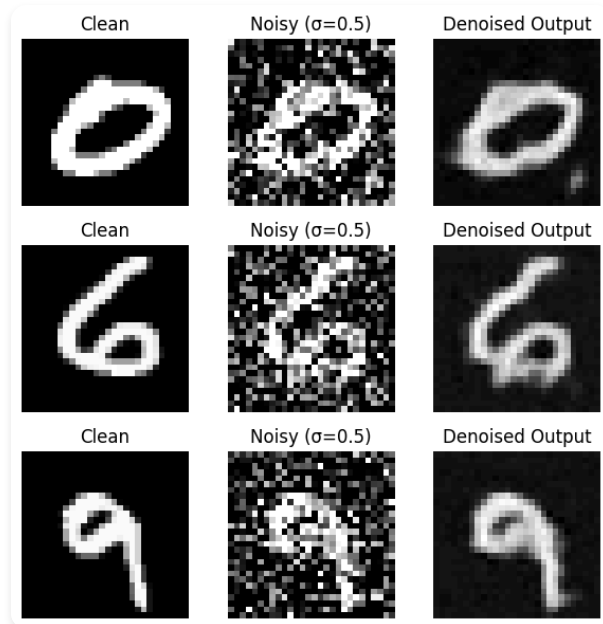
MNIST images with varying levels of Gaussian noise

### 1.2.1 Training

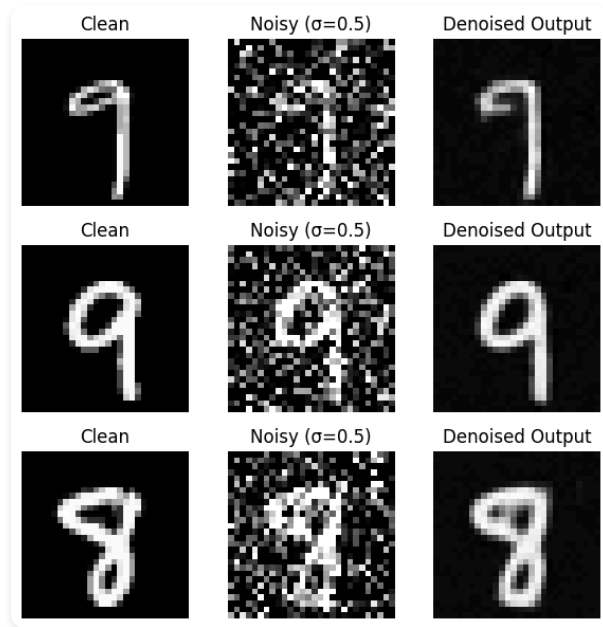
To train our model to perform denoising, I used the follow parameters recommended by the spec:

```
batch_size = 256
learning_rate = 1e-4
noise_level = 0.5
hidden_dim = 128
num_epochs = 5
```

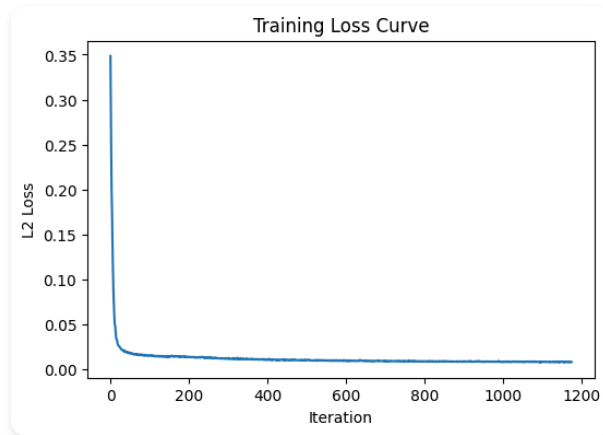
Next, we can load our training and testing datasets, making sure to shuffle before calling `DataLoader`. Using Adam optimizer and `MSELoss`, we can then train our UNet model to minimize the loss between the predicted denoised image and the original clean image. After training for 5 epochs, we can see that the training loss decreases steadily, indicating that our model is learning to denoise the images effectively. I think the most distinctive difference epochs 1 and 5 to my eye is that epoch 5 has clearer digits, whereas epoch 1's digits are slightly thick and appear kind of blurred.



Denoising after Epoch 1



Denoising after Epoch 5



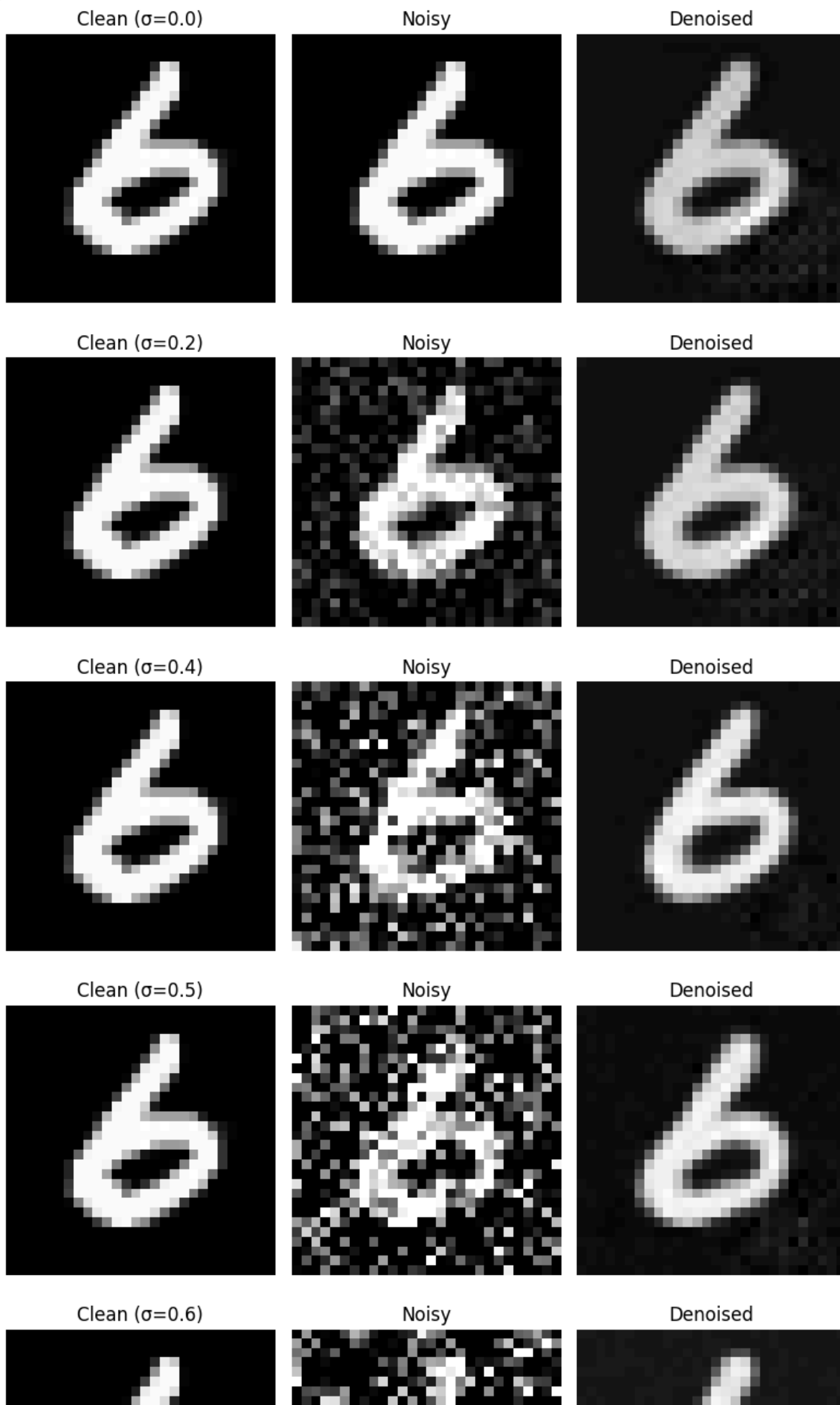
Training Loss Curve

## 1.2.2 Out-of-Distribution Testing

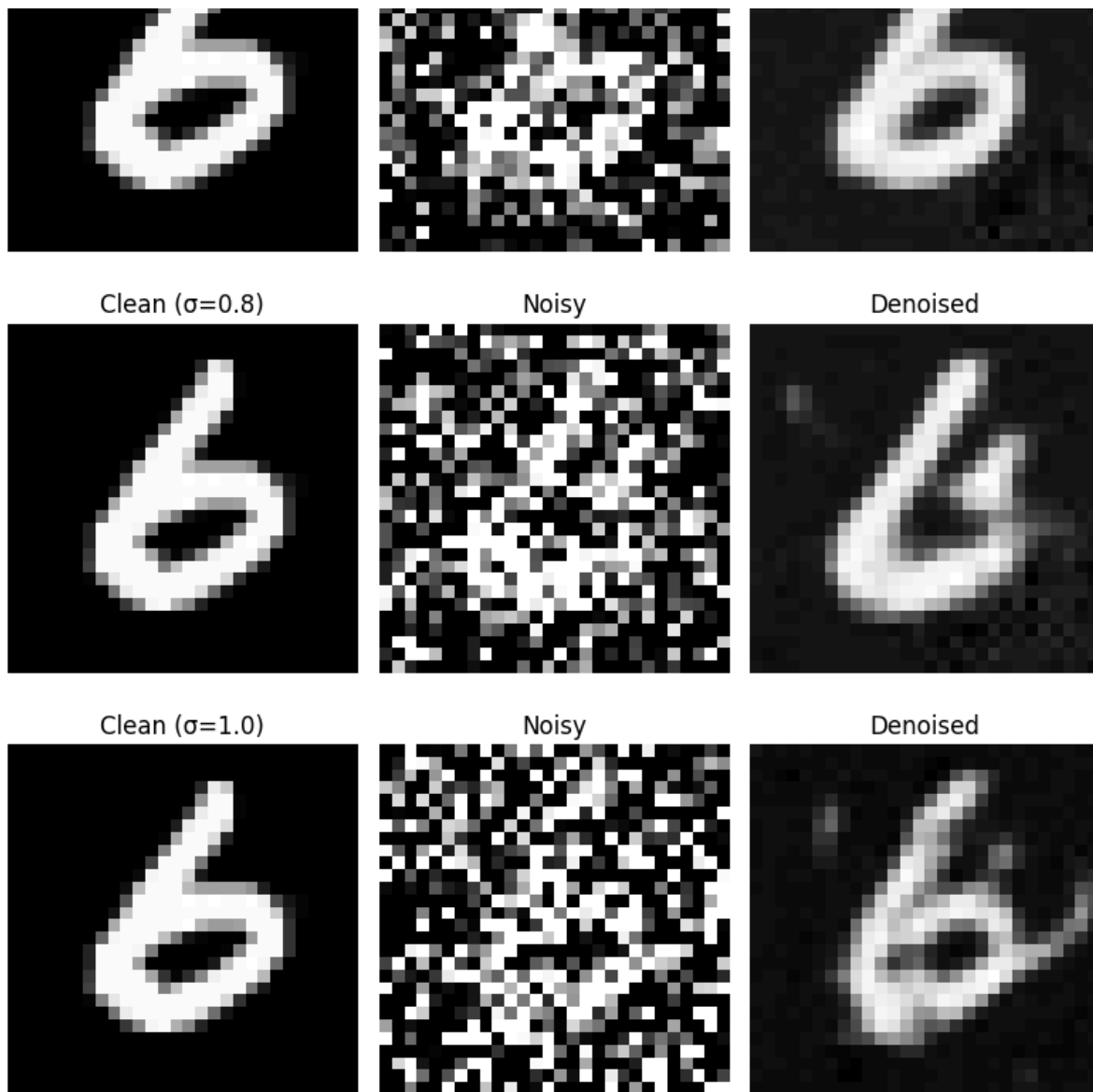
Next, to test how the denoiser performs on different sigma values that it wasn't trained for, I took an image from the test to add noise with sigma values of [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0] and then passed these noisy images through our trained UNet denoiser. Based on the results, it seems like the denoiser can perform

reasonably well up to noise levels of around 0.6, where the denoised image still clearly shows the correct digit. However, at a noise level of around 0.8 and 1.0 is when the output becomes less clear.









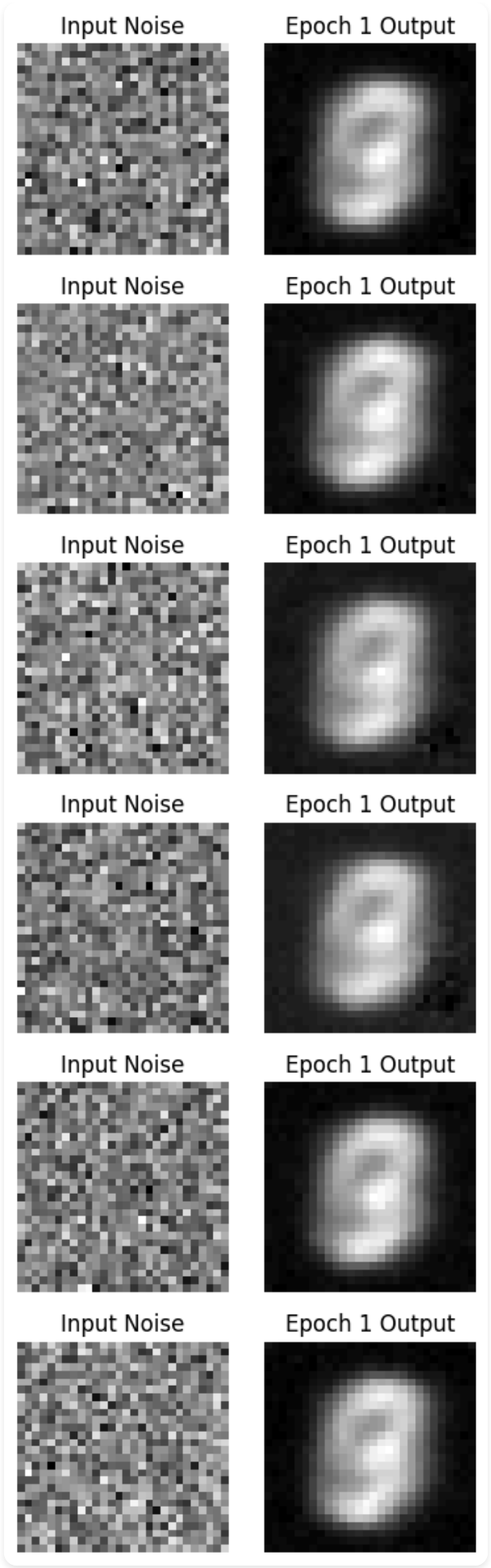
Out-of-Distribution Denoising Results

### 1.2.3 Denoising Pure Noise

Here, to train and sample on an input image of pure noise, I first used the `torch.randn_like` function again to create a random noise image with the same dimensions as our MNIST images. Then, I passed this noise image through our trained UNet denoiser to see what outputs it would produce again over 5 epochs. From there, I sampled 6 images to compare the input and output images. Interestingly, I notice a pattern where the output images from the inputs of pure noise all tend to resemble the digit 3 or 8, as a blurry combination of both. This may make sense intuitively when we consider our training image dataset contains

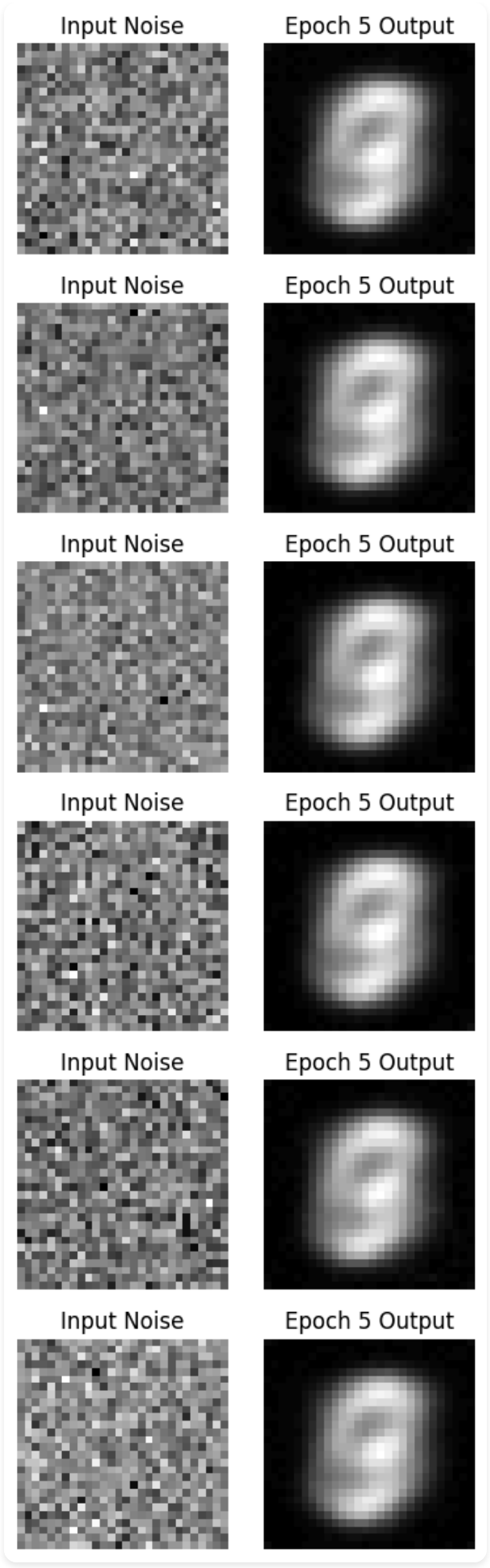
digits from 0-9. As the spec points out, since we are using MSE as our loss metric and the model learns to predict the point that minimizes the sum of squared distances to all training examples, then in the context of our MNIST training data, the model may tend to converge towards the most average representation of this data, which may involve loops similar to features from digits 3 and 8.





## Denoising Pure Noise (Epoch 1)





## Part 2: Training a Flow Matching Model

### 2.1 Adding Time Conditioning to UNet

To add time conditioning to our UNet, I add an FCBlock class to serve as an operator to transform a time value  $t$  into a higher-dimensional embedding vector. While some of our original UNet structure like ConvBlocks, DownBlocks, and UpBlocks remain the same, we can now inject a scalar  $t$  to effectively condition our model. In our `TimeConditionalUNet` class, we can modulate our unflatten operation by multiplying with our time embedding after passing  $t$  through our FCBlock. This way, the time information is incorporated into the feature maps as we reshape them back to  $7 \times 7$  dimensions. The rest of the UNet structure follows similarly to before, with downsampling and upsampling paths using DownBlocks and UpBlocks respectively, and concatenating corresponding encoder and decoder layers.

In our `time_fm_forward` process, we follow steps of sampling a timestep  $t$ , generating the intermediate point by following the linear interpolation formula B.3:

$$x_t = (1 - t)x_0 + tx_1 \quad \text{where } x_0 \sim \mathcal{N}(0, 1), t \in [0, 1]. \quad (\text{B.3})$$

Next, we subtract image  $x_0$  from  $x_1$  to calculate our true flow:

$$u(x_t, t) = \frac{d}{dt}x_t = x_1 - x_0. \quad (\text{B.4})$$

and then predict the flow using our time-conditioned UNet. From there, we compute the MSE loss between the true flow and predicted flow (true flow - predicted flow) to train our model:

$$L = \mathbb{E}_{x_0 \sim p_0(x_0), x_1 \sim p_1(x_1), t \sim U[0,1]} \|(x_1 - x_0) - u_\theta(x_t, t)\|^2. \quad (\text{B.5})$$

### 2.2 Training the UNet



Now that we have our time-conditioned model, we can follow Algorithm 1 to randomly pick an image from our training dataset, randomly choose a timestep  $t$ , add noise, and then run our denoiser to predict the flow at the noised image  $x_t$ . I used these parameters for training:

`batch_size = 64`

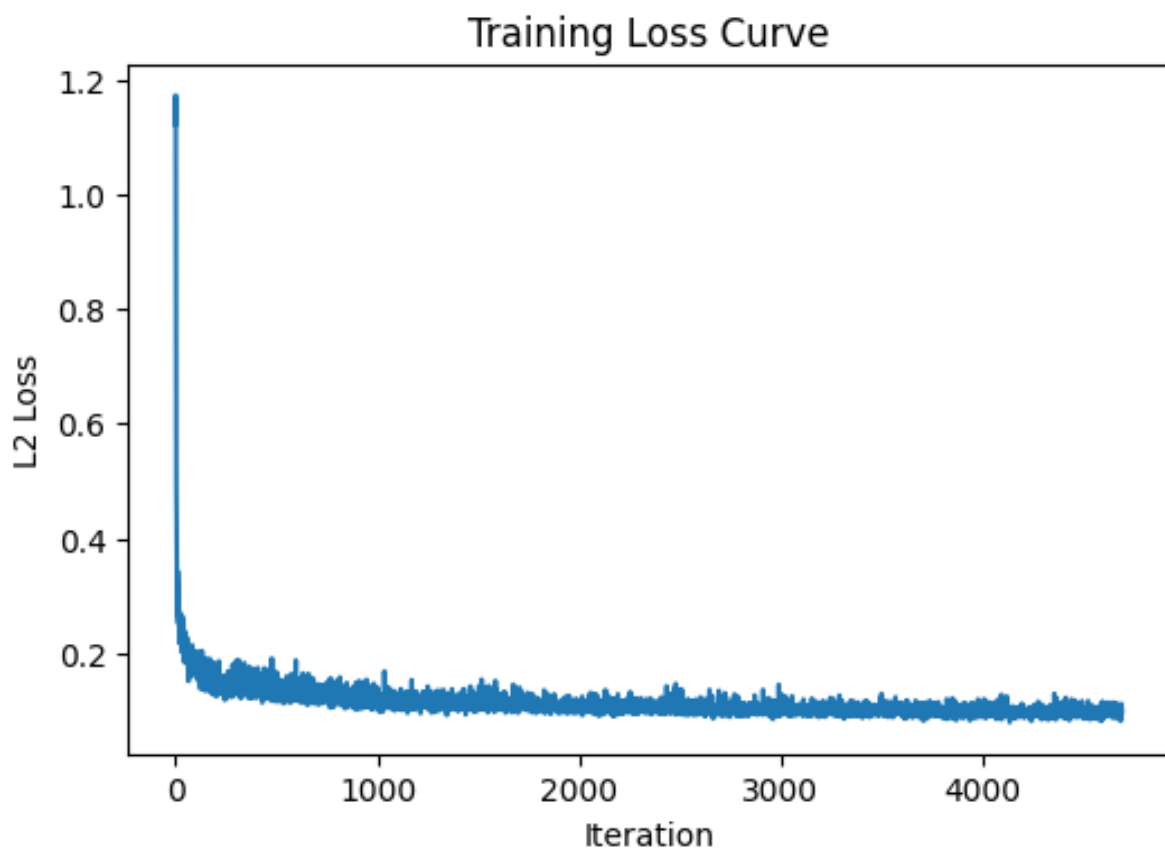
`learning_rate = 1e-2` (now with a scheduler + learning rate decay)

`hidden_dim = 64`

`num_epochs = 5`

`num_ts = 50`

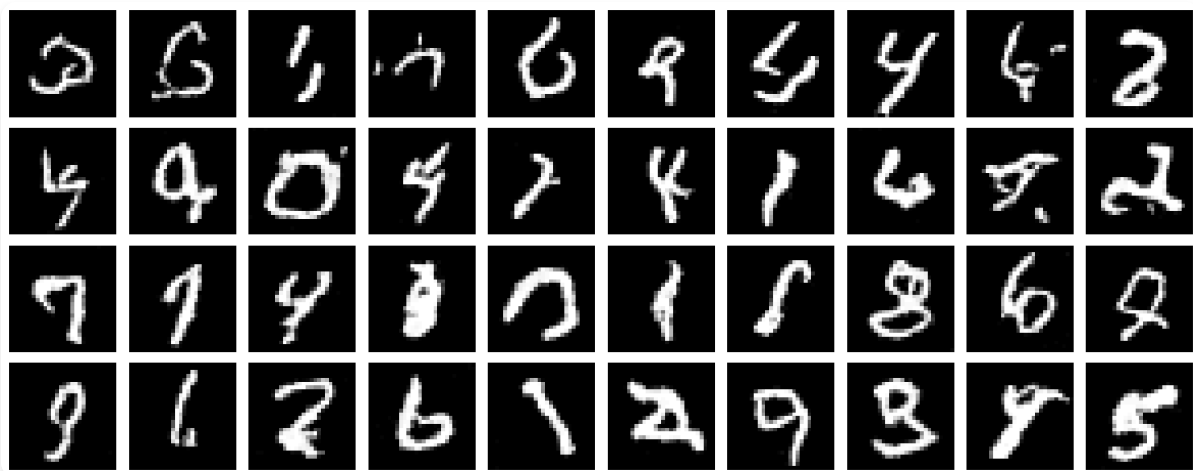
Below is the training loss curve plot for the time-conditioned UNet over the whole training process.



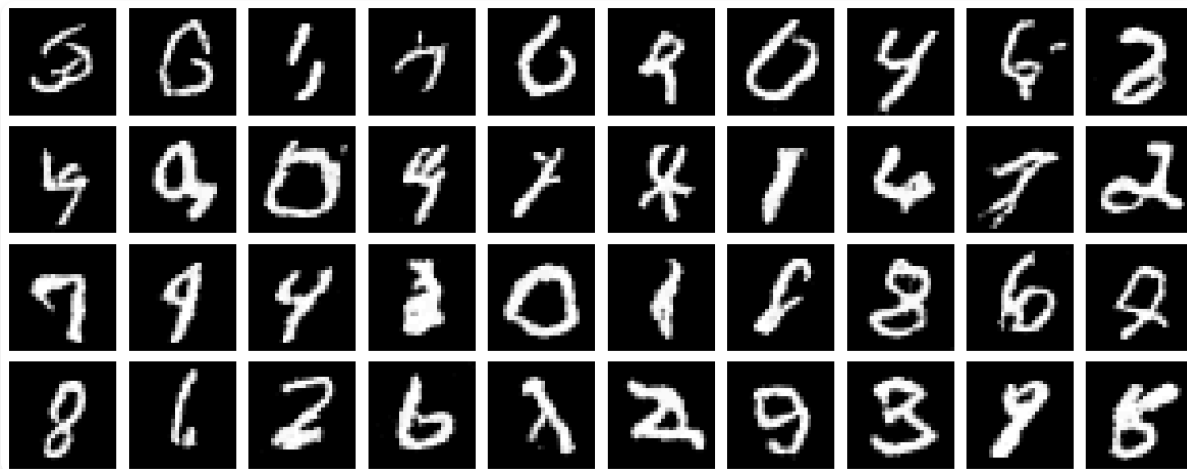
Training Loss Curve for Time-Conditioned UNet

## 2.3 Sampling from the UNet

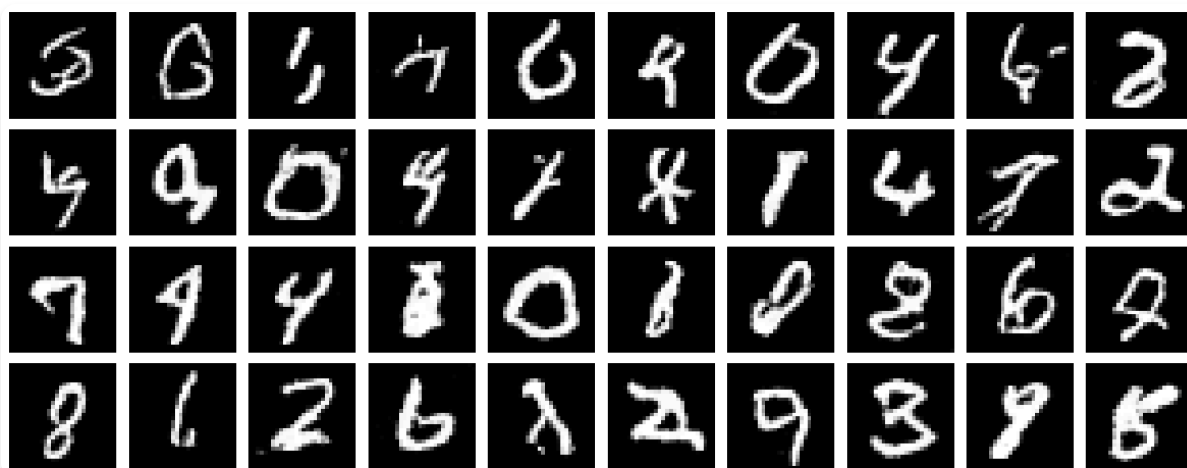
To sample from our trained time-conditioned UNet, we can follow Algorithm 2 from the spec. First, we start with a random noise image  $x_1$  sampled from a standard normal distribution. Then, we iterate from  $t = 0$  to  $t = 1$  in steps of size  $1/T = 1/\text{num\_ts}$  in my implementation, which is currently set to 50 timesteps. Finally, we update the current state by stepping in the direction of the predicted flow from our UNet scaled by the step size. Below are the results from sampling at epochs 1, 5, and 10, with a total of 40 images shown at each epoch. Just to comment on the result quality from each epoch, I think the digit 0 can be seen to become progressively more well defined and more clear over the course of iterating from epoch 1 to epoch 10. For example, in the first row in digit outputs in epoch 1's image, there are some numbers that appear very incomplete with just a couple disjointed strokes present. By epoch 10, I see fewer of disjointed handwriting and more complete digits that are easier to identify.



Sampling Results from Time-Conditioned UNet (Epoch 1)



Sampling Results from Time-Conditioned UNet (Epoch 5)



Sampling Results from Time-Conditioned UNet (Epoch 10)

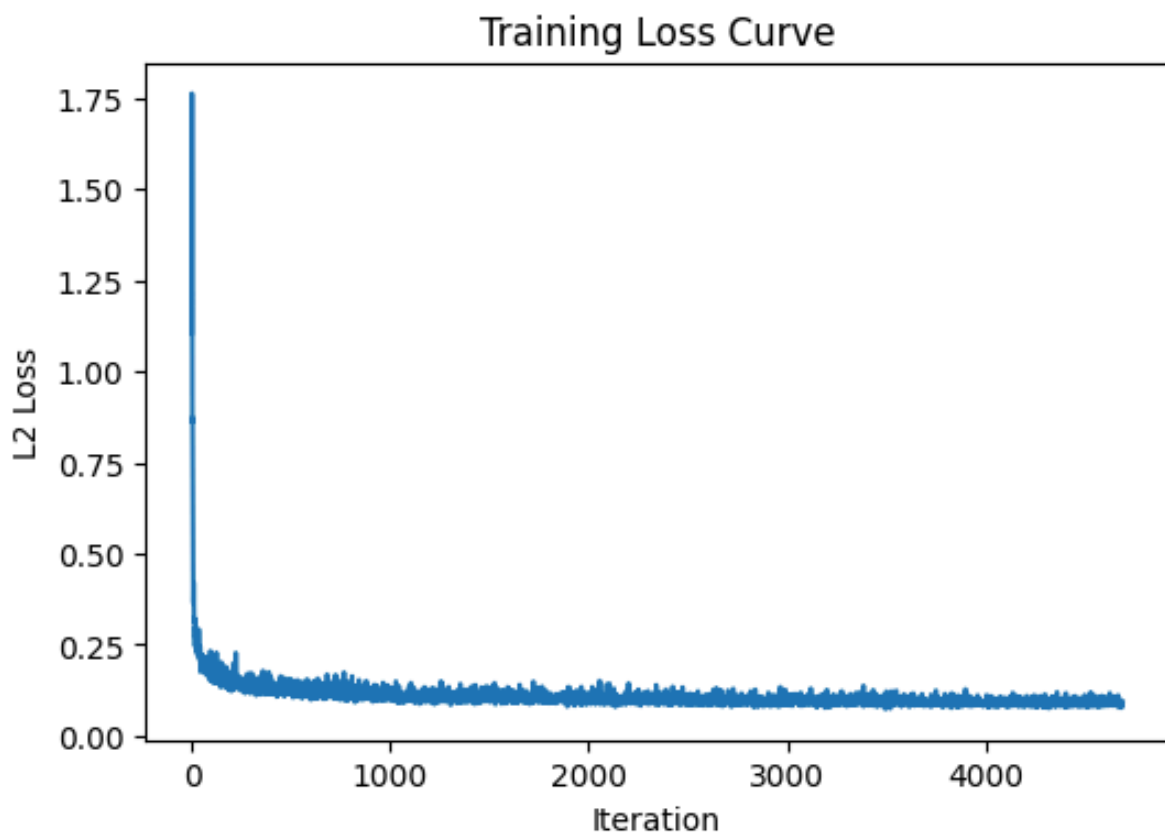
## 2.4 Adding Class-Conditioning to UNet

Here, to improve our model results, we can also add class-conditioning on top of our time-conditioned UNet to create a model conditioned additionally on the digit class 0-9. To do this, we add 2 more FCBlocks to process this class condition vector, and we also modify how we inject these embeddings into our UNet. Specifically, we multiply the class embedding and then add our time embedding in the unflatten operation, and we also do after our UpBlock during the upsampling

path. Following the spec's recommendation, we also one-hot encode our class labels to create a vector of size 10 for each digit class. In our `class_fm_forward` function, we also drop the class conditioning 10% of the time by setting this vector 0 instead to hopefully encourage the model to learn denoising without relying only on class information.

## 2.5 Training the UNet

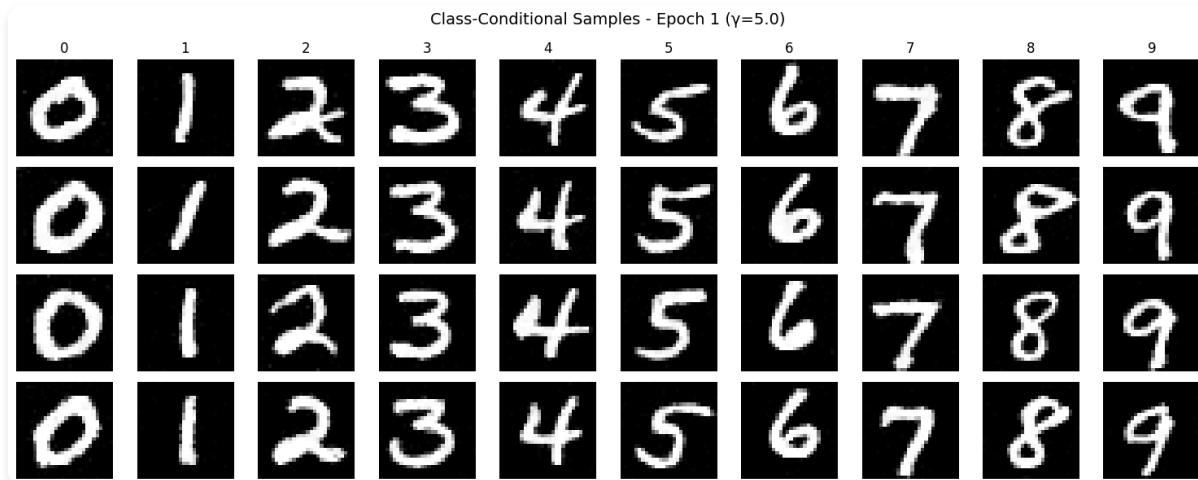
To train our class-conditioned and time-conditioned UNet, we can follow the same training process as in 2.2 along with the same parameters, but now with the addition of the 10 class labels. Below is the training loss curve plot for the class-conditioned UNet over the whole training process.



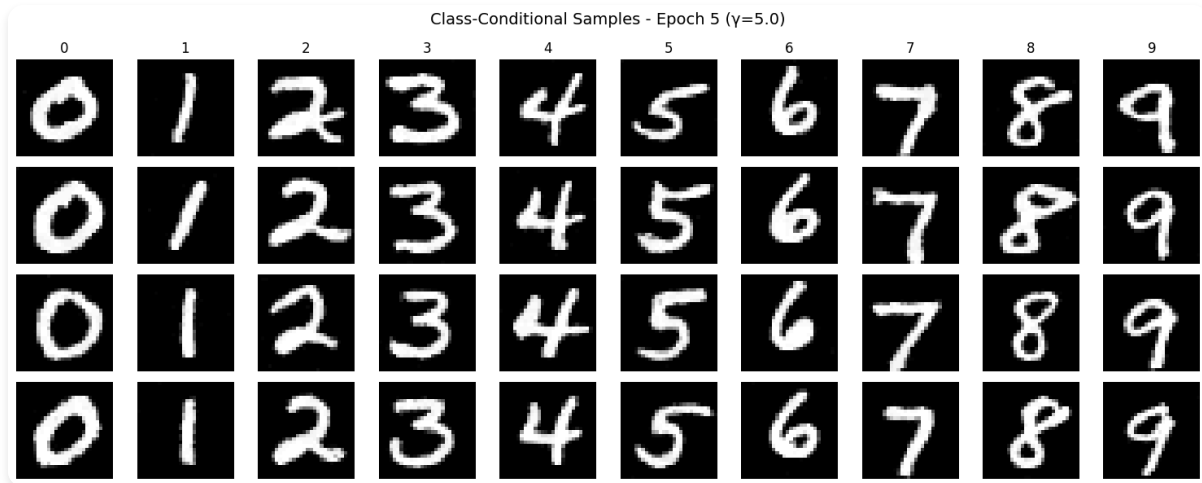
Training Loss Curve for Class-Conditioned UNet

## 2.6 Sampling from the UNet

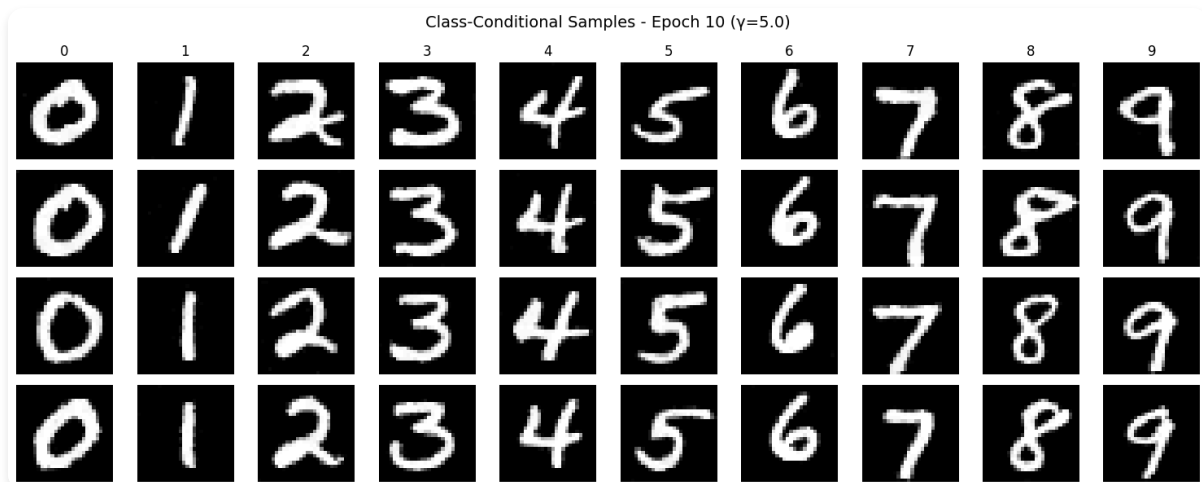
Next, we can sample from our trained class-conditioned UNet by following the same sampling procedure. Here, I sampled images for each digit class from 0-9 to see how well the model can generate images corresponding to each class and over only 10 epochs. Below are the results from sampling at epochs 1, 5, and 10, with a total of 10 images shown at each epoch for each digit class. While the output samples from the 3 epochs look similar to me, which may suggest that the model converged quickly, I do think that by epoch 10, certain digits like the sampled digit number 7 look cleanest in the final epoch, where there's fewer stray pixels and the digit is more clearly defined.



Sampling Results from Class-Conditioned UNet (Epoch 1)



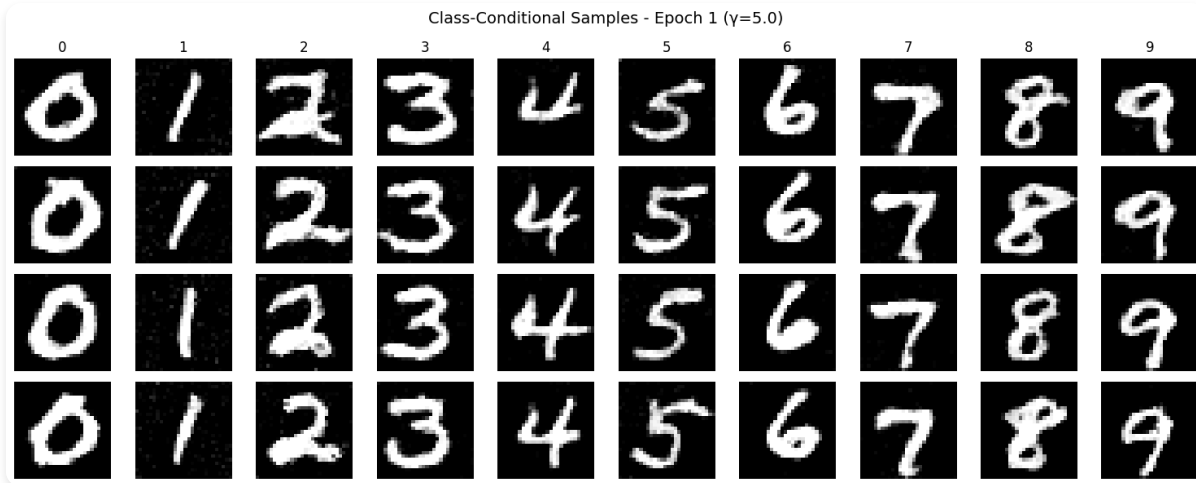
Sampling Results from Class-Conditioned UNet (Epoch 5)



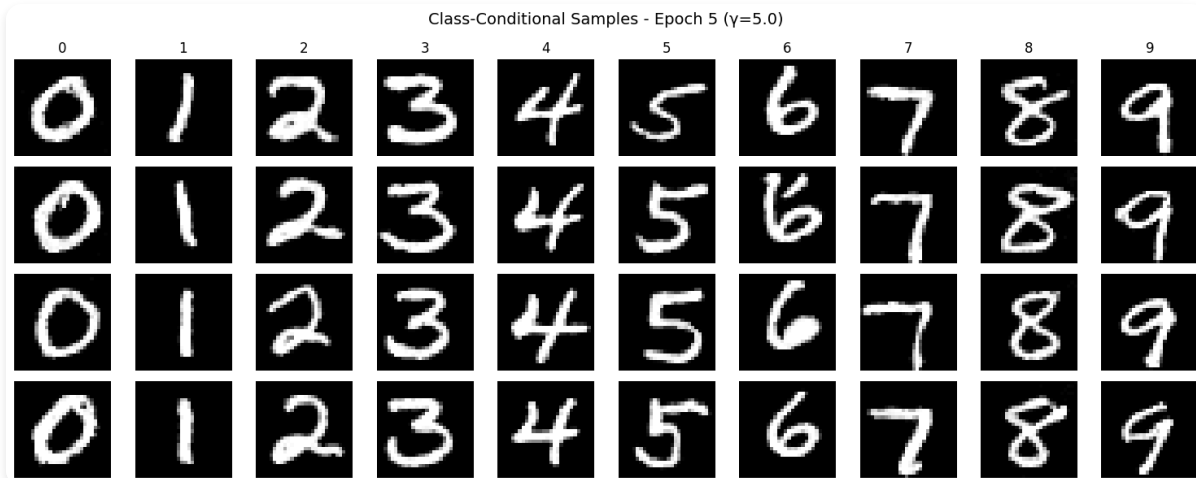
Sampling Results from Class-Conditioned UNet (Epoch 10)

To try this without the learning rate scheduler, I changed my learning rate parameter to  $3e-3$  because this value would be between the start and end learning rates of the scheduler before. As a result, I hope a smaller but fixed learning rate to compensate for lack of decay may help the model converge better. Below, I show the 3 sampling results at epoch 1, 5, and 10 so that we may compare against the UNet with the learning rate scheduler, and I also included a comparison of the training loss curves, where we can see the difference in model convergence.

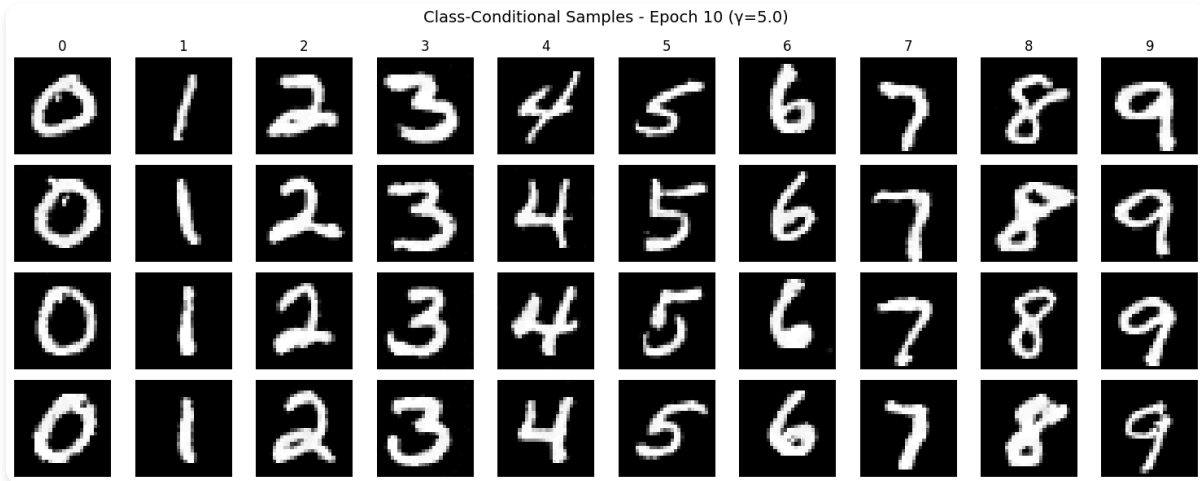
Overall, I think the results are actually fairly close between the two models with a learning rate scheduler and without!



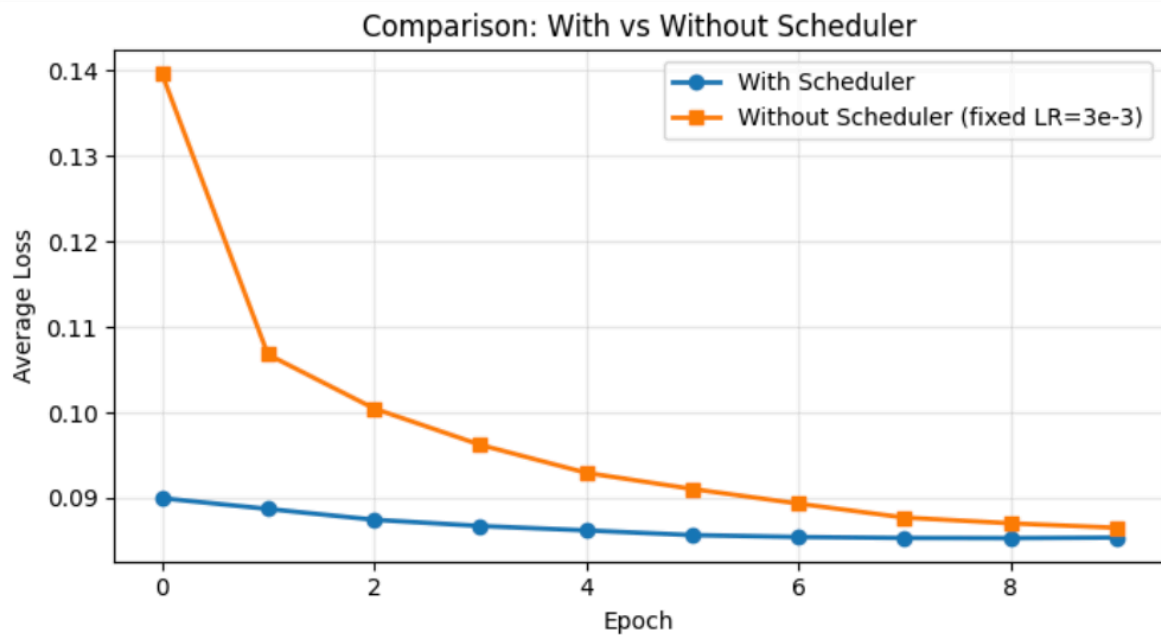
No Learning Rate Scheduler Sampling Results from Class-Conditioned UNet (Epoch 1)



No Learning Rate Scheduler Sampling Results from Class-Conditioned UNet (Epoch 5)



No Learning Rate Scheduler Sampling Results from Class-Conditioned UNet (Epoch 10)



Training Loss Curve for Class-Conditioned UNet Comparison